

Feedback File Definition

Harald Anlauf, Andreas Rhodin

April 1, 2026

Contents

1. Overview	6
2. Feedback File Definition	7
2.1. Global Attributes	7
2.2. Dimensions	8
2.3. Data	9
2.3.1. Report Headers	9
2.3.2. Report Bodies	11
2.3.3. RADAR observation type specific table	11
2.3.4. Verification Data	12
3. Definition of Data Items	13
3.1. Observation and Code Types	13
3.2. Report and observation status	15
3.3. Quality check flags	16
3.4. Observation variable and level types	17
3.5. General and individual cloud group	19
3.6. Level significance	20
3.7. Specification of Instruments and computational methods	20
3.8. Observation operator specific settings	22
3.9. 1DVAR Surface type, cloud flags and processing flags	23
3.10. Model surface characteristics	24
3.11. Soil types	25
3.12. Observation quality information	25
3.13. Specification of verification runs	26
4. Migration of COSMO VOF format to NetCDF format	27
4.1. File Header	27
4.2. Report Header	28
5. Fortran 90 interface	30
5.1. Tables	30
5.1.1. Data type definitions (mo_t_table)	30
5.1.2. Feedback file tables (mo_fdbk_tables)	32
5.2. NetCDF interface	32
5.2.1. Data type definitions (mo_t_netcdf_file)	32
5.2.2. Feedback file interface (mo_fdbk)	35
5.2.3. Feedback file I/O handling (mo_fdbk_io)	40
5.2.4. Feedback file output for radiances (mo_fdbk_rad)	43
5.3. Test facilities and 3D-Var interface	44
5.3.1. Test program (test_feedback)	44

5.3.2. Test program (test_feedback_read)	45
5.3.3. Test program (test_fdbk_rad)	45
5.3.4. 3d-Var interface (mo_fdbk_3dvar)	45
6. Feedback File Example (ncdump -h)	45
A. Filename conventions for feedback files	50

List of Tables

1. Changes to this document	5
2. NetCDF variables for report headers	10
3. NetCDF variables for report bodies	11
4. Specific NetCDF variables for RADAR observations	12
5. NetCDF variables for verification data	12
6. Observation Types	13
7. Oceanographic Observation Types	13
8. Observation Code Types	15
9. Observation or report status values	15
10. Report or Observation Flags	16
11. Observation and level variable numbers	19
12. general cloud group bit positions	19
13. individual cloud group bit positions	20
14. level significance for TEMP/PILOT.	20
15. radiosonde type (NRARA), for other values see WMO common code table C2	21
16. tracking technique (NSASA), for other values see WMO common code table C7	21
17. satellite derived wind computation method	21
18. type of measuring equipment used (NA4, 002003)	22
19. solar and infrared radiation correction (NSR, 002013)	22
20. Observation operator flag (observation type dependent)	22
21. surface types consistent with 1d-Var	23
22. cloud flag	23
23. TOVS specific flag	24
24. Cloud type according to NWC SAF	24
25. model surface characteristics	25
26. type of soil in the retrieval point	25
27. aircraft phase, radiances fov, gpsro pcd	25
28. aircraft roll angle	26
29. type of verification run	26
30. class of verification run	26
31. specification of the verification data	27

Changes to this document

Date	Version	Change
2026-04-01		Add codetype for EDP from rof file (EDP_ROF).
2026-01-27		Add codetype for SWH from satellite (OCE_SWH).
2025-11-11		Add codetype for AOD measurements; add varno for AOD.
2025-11-04		Filename conventions for feedback files.
2025-08-28		Add varno for significant wave height (VN_SWH).
2025-07-04		RADAR: add varnos for RZDR, RKDP, RCC (polarized measurements); LWD, IWD, NCL, NCI, VMDL, VMDL (hydrometeor retrievals).
2025-06-30		PILOT: Update for doppler wind lidar observations. Add body variable average . PILOT instype refers to WMO table 0 02 006.
2024-12-10		Update description: varno Z is geopotential height.
2024-11-06		Add codetype for SSNOW (snow observation from SYNOP), SYTEMP, SYTOWR (TEMP-derived/tower-derived surface report). Add codetype for SRSCD_ROF, CWS_ROF, LDPCD_ROF (raw observation file, equivalents of SRSCD, CWS, LDPCD) Add codetype for OCE_SEAICE (sea ice product).
2024-05-03		Add varno for sea ice (area) fraction (VN_FR_ICE).
2024-04-09		Add varnos for greenhouse gases (CO2, CH4, N2O).
2024-03-21		Add obstype MWR (ground-based microwave radiances). Add codetype for DIAL (differential absorption lidar). Add oceanographic obstypes (new table) and codetypes (common table). Document optional header entry 'fr_land' (SYNOP, DRIBU). Document optional header entry 'sso_stdh' (SYNOP).
2024-01-03		Document optional body entries 'dlat', 'dlon': "Drifting" latitude/longitude. New header entry 'center_id': Processing center ID (GPSGB, GPSRO). New header entry 'sat_class': Satellite classification (GPSGB, GPSRO). New header entry 'prn': transmitter ID (GPSRO).
2023-08-30		Add variable numbers for SALINITY, SWPT, SWT, SSH, SLA (ocean). Update table for TOVS specific flags.
2022-12-20		Add codetype for RAMAN lidar, CWS (citizen weather stations), TAMDAR, MWR (microwave radiometers). Add variable numbers for CEIL, SOILM, NSOILM, MIXR. Replace tables for TOVS specific flags and surface types.
2021-06-17		Add roll angle quality as AIREP level_sig.
2020-09-21		Add dummy observation codetypes, variable numbers for testing purposes. Add NWC SAF cloud types. Add soil types.
2019-10-25		Add variable numbers for cell-based VIL, number of objects. Add codetype for Additional Land Surface Data.
2019-08-16		Add codetype for CMAN (coastal marine automated network) data. Add codetypes for SWIS (road weather information system) and CAR data. Add codetypes for tower profile data and for ground-based lightning data. Add table with TOVS specific flags.

2019-04-02		Add codetypes for high-resolution BUFR TEMPs (land/ship/drop/descent). Add variable number for height of sensor above ground.																					
2019-01-28		Add section on observation operator specific settings.																					
2018-11-22		Update AMV retrieval types.																					
2018-04-24		Update of feedback file definition tables. Removed obsolete entry <code>flg_1dvar</code> .																					
2017-04-14	1.02	Increase the version number of feedback files to 1.02 : Fixed a bug in the COSMO and MEC CDFIN input routines (relevant for LETKF ekf-files and MEC). Work around this bug when reading fof-files (relevant for COSMO-LETKF). In the COSMO and MEC CDFIN input routines and in the forward operator (first guess) the components u and v of the wind were calculated in the rotated coordinate system of the model and written to the fof-files in this form. The same transformation was done in the MEC CDFIN input routines but not in the calculation of the model equivalents. This resulted in the following output in the fof-, ekf-, and verification-files: <table> <tr> <td>observation</td><td>fof-file from COSMO</td><td>in model rotated coordinates</td></tr> <tr> <td>first guess</td><td>fof-file from COSMO</td><td>in model rotated coordinates</td></tr> <tr> <td>observation</td><td>ekf-file from LETKF</td><td>in model rotated coordinates</td></tr> <tr> <td>first guess</td><td>ekf-file from LETKF</td><td>in model rotated coordinates</td></tr> <tr> <td>analysis</td><td>ekf-file from LETKF</td><td>in model rotated coordinates</td></tr> <tr> <td>observation</td><td>ver-file from MEC</td><td>in model rotated coordinates</td></tr> <tr> <td>model equivalents</td><td>ver-file from MEC</td><td>in geographic coordinates</td></tr> </table> After the fix (indicated by feedback file version 1.02) u and v will always be given correctly in geographic coordinates. The LETKF analyses and forecasts (model equivalents) are not affected (always correctly calculated and represented in rotated model coordinates in the GRIB file).	observation	fof-file from COSMO	in model rotated coordinates	first guess	fof-file from COSMO	in model rotated coordinates	observation	ekf-file from LETKF	in model rotated coordinates	first guess	ekf-file from LETKF	in model rotated coordinates	analysis	ekf-file from LETKF	in model rotated coordinates	observation	ver-file from MEC	in model rotated coordinates	model equivalents	ver-file from MEC	in geographic coordinates
observation	fof-file from COSMO	in model rotated coordinates																					
first guess	fof-file from COSMO	in model rotated coordinates																					
observation	ekf-file from LETKF	in model rotated coordinates																					
first guess	ekf-file from LETKF	in model rotated coordinates																					
analysis	ekf-file from LETKF	in model rotated coordinates																					
observation	ver-file from MEC	in model rotated coordinates																					
model equivalents	ver-file from MEC	in geographic coordinates																					
2012-09-06		Changes for radar operator.																					
2012-01-26		Implement variables for radar operator, VN_VLS: line of sight wind.																					
2012-01-04		Add comments, update f90 interface documentation.																					
2011-12-05	1.01	Change variable 'phase' from BYTE to SHORT, use for GPSRO PCD as well. Write variable 'plev' for AIREPs as well. New body entry 'accuracy': accuracy from data provider. Use optional variable 'plevel' for PILOTs as well. Write data base time 'time_dbase' to NetCDF feedback file. New optional parameters 'ierr', 'fill') for subroutines <code>get_var_real_2</code>, <code>read_fdbk_veri</code>: error return parameter, value to replace with fillvalue. Modifications for writing COSMO feedobs files. Change 'level_sig' to SHORT to hold WMO instrument numbers (instead of RTTOV). New routines 'write_veri', 'write_fdbk_var', add error handling. New flag values: codetype OC_WP_JP = 134: Japanese wind profilers, 'varno' parameter VN_FLEV: nominal flight level for Aircrafts, status flag ST_OBS_ONLY: observations only, no model equivalent available, VE_ENS_MEAN_OBS: ensemble mean taken in observation space,																					

		FL_NO_BIASCOR: no bias correction available, FL_NO_OBS: no observations in report, FL_OPERATOR: observation operator not applicable, VE_MEMBER: generic value for complete set of ensemble members.
2011-02-08		define additional parameters to write COSMO feedobs files. new flags FL_NO_BIASCOR, FL_NO_OBS. make variable 'sun_zenith' mandatory for all observation types. use optional variable 'tracking' also for AIREP observations. new Fortran 90 interface module mo_fdbk_io.
2010-07-28		add overview section, update f90-interface description.
2010-06-08		meaning of veri_initial_date and veri_forecast_time clarified.
2010-01-12	1.00	changes for RADAR data: new header entries range bin size, varno fallback, make varno optional. make level, level_typ, level_sig optional. define radar specific table, entries: radar_azimuth, radar_elevation, radar_range. new dimension and attribute: d_radar, n_radar.
2009-11-12	0.99	radiances: use phase for Field Of View index. new body entry sat_zenit : satellite zenith angle. new body entry sun_zenit : sun zenith angle.
2009-05-04	0.99	type of variable instype changed from byte to short.
2009-04-23	0.99	new variables: mdlsfc, qual, plevel. removed variables: mdlsf, pcc, snr.

Table 1: Changes to this document

1. Overview

Feedback files hold information on observations and their usage in the data assimilation system:

- Type of observation
- Coordinates of observation
- Specification of the measurement instrument or station
- Observed values
- Background values used in the data assimilation
- Analysed value
- Usage in the assimilation system (i.e. used, rejected, etc.)
- Bias correction used (if applicable)
- Weight in variational quality control (if applicable)
- Optionally: Values of forecasts with different lead times
- ...

Feedback files are currently used to gather information for verification and innovation statistics in the global data assimilation system (GME, ICON) and shall be used in the regional data assimilation (COSMO, ICON-LAM) as well. They are also used in the COSMO ensemble data assimilation system (KENDA) to pass information on observations and first guess (taken at the appropriate time) from the COSMO model to the LETKF.

This document describes the format and content of the NetCDF feedback files (Sections 2 and 3), provides hints for the migration from the COSMO VOF files to this format (Section 4), a description of the Fortran 90 interface routines to read and write these files (Section 5), and an example ncdump (Section 6).

2. Feedback File Definition

2.1. Global Attributes

Global attributes following the NetCDF Climate and Forecast (CF) Metadata Conventions:

title (character string, len=28)

A succinct description of what is in the dataset. Model name and string 'Verification Data':

'COSMO Verification Data'

institution (character string, len=24)

Specifies where the original data was produced. In general:

'German Weather Service'

source (character string, len=16)

The method of production of the original data. If it was model-generated, source should name the model and its version, as specifically as could be useful. For instance:

'COSMO Version Y.Z'

history (character string, len=multiple of 80)

Provides an audit trail for modifications to the original data. Well-behaved generic netCDF filters will automatically append their name and the parameters with which they were invoked to the global history attribute of an input netCDF file. We recommend that each line begin with a timestamp indicating the date and time of day that the program was executed:

```
'2007-11-23 01:23 COSMO 2007112300 deterministic fc '
'2007-11-23 01:23 COSMO 2007112300 deterministic fc '
'2007-11-23 02:24 COSMO 2007112300 Ensemble-member 001'
'2007-11-23 03:45 COSMO 2007112300 Ensemble-member 002'
'2007-11-23 02:24 LETKF 2007112300 '
```

Each entry uses 80 characters, separated by newline characters.

The following CF recommendations are not used:

references

Published or web-based references that describe the data or methods used to produce it.

comment

Miscellaneous information about the data or methods used to produce it.

In addition to the CF recommendations the following global attributes are set:

file_version_number (character string, len=5)

'01.02'

Feedback file specific attributes:

n_hdr (integer)

Number of report entries actually used. May be smaller than the allocated dimension d_hdr.

n_body (integer)

Number of body entries actually used. May be smaller than the allocated dimension `d_body`.

n_radar (integer)

Number of RADAR observation specific entries actually used. May be smaller than the allocated dimension `d_radar`.

verification_ref_time (integer, hhmm)

verification_ref_date (integer, yyyymmdd)

Verification reference time and date.

In case of a forecast reference time is the start of the forecast period.

In case of the nudging scheme reference time is the end of the assimilation period.

In case of 3D-Var or LETKF reference time is the analysis time.

verification_start (integer, minutes)

verification_end (integer, minutes)

Start and end of the verification period with respect to verification reftime.

resolution (float(2))

Model resolution in degree, separately for the x- and y-dimension.

domain_size (integer(3))

COSMO: nx, ny, nz

GME: ni, ni, nz

ICON: nxny, nxny, nz

For the COSMO model:

pole_lat_lon (float(2))

lower_left_lat_lon (float(2))

upper_right_lat_lon (float(2))

Coordinates of the rotated pole and the edges of the model domain (degree).

2.2. Dimensions

name	description
<code>d_hdr</code>	number of reports in the file.
<code>d_body</code>	number of observations.
<code>d_radar</code>	size of RADAR observation specific table. The allocated size of <code>d_hdr</code> , <code>d_body</code> and <code>d_radar</code> may be larger than the number of data items actually stored. The latter is given in the global attributes <code>n_hdr</code> , <code>n_body</code> and <code>n_radar</code> .
<code>d_veri</code>	number of verification runs. This dimension is the unlimited dimension, so that further verification runs can be added.
<code>d_2,d_3</code>	Arrays of length 2 or 3
<code>char10,char12,char64</code>	length of character strings

2.3. Data

Data is either of dimension (d_hdr) for the report header or of dimension (d_body) for data related to each observation, or of dimension (d_veri, d_body) for verification data. There are observation operator specific tables (RADAR operator so far) of size (d_radar).

2.3.1. Report Headers

Data is of dimension (d_hdr) and provides information related to each report (single level report, multi level report, satellite field of view).

Data may be optional (cf. column 'optional' in the table below). Optional data may be restricted to a model (COSMO, GME, ICON), to the assimilation scheme (3DVAR, LETKF) or to certain observation operators (TEMP, PILOT, ...).

name	type	optional	units	description
i_body	int			Index of the data in the corresponding report body. For example: 1: for the first report $n + 1$: for the second report if the first one consists of n observations.
l_body	short			Number of observations in the corresponding report body. n for the first report in the example above.
n_level	short			Number of different levels in the report body.
i_spec	int	RADAR		Index of the data in the corresponding report type specific table. For example: 1: for the first report of a given obstype $n + 1$: for the second report of this obstype if the first one consists of n observations.
l_spec	short	RADAR		Number of observations in the corresponding report specific table n for the first report in the example above.
data_category	short		WMO	BUFR 4 data category.
sub_category	short		WMO	BUFR 4 international data sub-category.
center	short		WMO	Station processing center.
sub_center	short		WMO	Station processing sub-center.
obstype	byte		Table 6	Observation (report) type.
codetype	short		Table 8	Observation code type.
ident	int			Station ID as integer. Not applicable for aircraft reports or ship TEMP reports.
			WMO C-2	Satellite ID for satellite observations.
statid	char			Station id in character form. (10 char)
lat	float		degree	Latitude of observation.
lon	float		degree	Longitude of observation.
time	short		min	Observation time minus reference time.
time_nomi	short		min	Nominal (synoptic) observation time minus reference time.
time_dbase	short		min	Data base time minus reference time. At DWD decoding time. Used to monitor/simulate data available within a certain cutoff time.
z_station	short		m	Station height.
z_modsurf	short		m	Model surface height.
sun_zenit	float		degree	Sun zenith angle.

r_state	byte		Table 9	Status of the report.
r_flags	int		Table 10	Quality check bit flag.
r_check	byte		Table 10	Check which caused rejection of the report. One value out of r flags.
sta_corr	byte			Station correction indicator (1: yes; 0: no).
index_x	short			model index x of grid point to which the report is assigned.
index_y	short			model index y of grid point to which the report is assigned.
mdlsfc	byte		Table 25	Model surface characteristics.
instype	short		Table 15	TEMP: Radiosonde type
			WMO C-8	GPSRO, RAD: Classification of Satellite instruments (AMSUA, AMSUB).
			WMO 002 149	BUOY: type of data buoy.
			WMO 002 001	SYNOP, AIREP: type of station.
			WMO 002 006	PILOT: Upper air remote sensing instrument type.
sat_class		GPSRO	WMO 002 020	Satellite classification (GNSS).
prn		GPSRO	WMO 001 050	Transmitter ID.
center_id		GPSGB		Processing center ID.
		GPSRO		
retrtype	short	SATOB	Table 17	Station retrieval type.
tracking	byte	TEMP	Table 16	Tracking technique.
		PILOT		
		AIREP		
meas_type	byte	TEMP	Table 18	Type of measuring equipment used.
		PILOT		
rad_corr	byte	TEMP	Table 19	Solar and infrared radiation correction.
phase	byte	AIREP	Table 27	Aircraft phase: Bit flags from BUFR report.
		SCATT		
		GPSRO		
		RAD		Radiances, Scatterometer: Field of View index.
flg_cld	byte	RAD	Table 22	Bit pattern for 1DVAR cloud flag.
surftype	byte	RAD	Table 21	Bit pattern for surface type.
sat_zenit	float	RAD	degree	Satellite zenith angle.
varno_back	short	RADAR	Table 11	Type of the observed quantity. Used in case that all observations of the report are of the same type. In that case body entry varno may be not present.
vnyquist	float	RADAR	m	Nyquist frequency.
spec_r_flags	int	RADAR		observation type specific flags.
fr_land	float	SYNOP	(0–1)	(Model) land fraction.
		DRIBU		
sso_stdh	float	SYNOP	m	(Model) SSO standard deviation.
obs_id	int	3DVAR		Unique observation Id in the assimilation program. Remains unchanged if a different number of processors is used.
source	byte	3DVAR		Input file number.
record	int	3DVAR		Record number of the report in the original input (BUFR) file.
subset	short	3DVAR		Subset number of the report in the original input (BUFR) file.
dbkz	short	3DVAR		DWD data base id.
index_d	byte	GME		model diamond index to which the report is assigned.

Table 2: NetCDF variables for report headers

2.3.2. Report Bodies

Data fields with dimension (d_body) for data related to individual observations:

name	type	optional	units	description
obs	float			Bias corrected observation.
bcor	float			Bias correction (corrected - observed value).
e_o	float			Observational error.
state	byte		Table 9	Status of the observation.
flags	int		Table 10	Bit flag table.
check	byte		Table 10	Check which caused rejection of the observation. One value out of flags.
qual	short		% Table 28	Observation confidence from data provider: AMV: 0=very bad, 100=very good. Aircraft: roll angle. Wind profiler: signal to noise ratio.
accuracy	float	PILOT GPSRO	as obs	Accuracy of observation from data provider, used for some wind profilers.
plevel	float	GPSRO RAD PILOT AIREP	Pa	Nominal height for observations which do not have a vertical pressure coordinate.
varno	short	not RADAR	Table 11	Type of the observed quantity.
level	float	not RADAR		Value of observation level. (In general pressure in Pa).
level_typ	short	not RADAR	Table 11	Type of level information.
level_sig	short	TEMP PILOT SYNOP AIREP RAD	Table 14 WMO 008 002 WMO 002 064 WMO C-8	Level significance. Level significance for individual and general cloud type. Aircraft roll angle quality. Instrument type for radiances.
azimuth	float	GPSGB GPSRO WLIDAR	degree	azimuth of line of sight.
dlat	float	GPSRO GPSGB TEMP RADAR	degree	Effective (“drifting”) latitude of observation. Nominal (reported) latitude of observation.
dlon	float	GPSRO GPSGB TEMP RADAR	degree	Effective (“drifting”) longitude of observation. Nominal (reported) longitude of observation.
spec_index	int	RADAR		Index of observation for the case of sparse coverage.
average	short	PILOT	sec	Averaging period of observation.

Table 3: NetCDF variables for report bodies

2.3.3. RADAR observation type specific table

A specific table is defined for volume radar data in order to allow a sparse storage of the observations and to efficiently describe the underlying coordinate system.

One RADAR report holds all data for a nominal elevation angle. Thus the observations are specified as a function of two coordinates only: azimuth and distance. Entries in the observation body are only stored for

valid data and the relation to the complete data set is given by variable `spec_index`: i.e. the bin number within the ray.

The entries of the RADAR observation type specific table are related to the header table entries by the variables `i_spec` and `l_spec`. The size of the table corresponds to the number of azimuth entries. Each entry holds additional information on the coordinates:

name	type	optional	units	description
<code>radar_azimuth</code>	float	RADAR	degree	azimuth of ray.
<code>radar_elevation</code>	float	RADAR	degree	elevation of ray (may differ from nominal value).
<code>radar_nrange</code>	short	RADAR		range of ray (may be less than the maximum value).
<code>radar_range_start</code>	float	RADAR	m	distance of first bin from station.
<code>radar_drangle</code>	float	RADAR	m	distance between bins.
<code>radar_nbody</code>	short	RADAR		number of body entries for this ray.

Table 4: Specific NetCDF variables for RADAR observations

2.3.4. Verification Data

Verification data is stored in the variable `veri_data` with dimensions (`d_body`, `d_veri`). The content of `veri_data` (modelled quantities or estimates of the errors of the modelled quantities) and the type of the verification run (analysis, forecast, ...) is specified in the accompanying variables of dimension (`d_veri`):

name	type	units	description
<code>veri_data</code>	float		Modeled quantity or estimates of the error of the modeled quantity.
<code>veri_model</code>	char(10)		Model used for verification, e.g. 'COSMO', 'GME'
<code>veri_run_type</code>	byte	Table 29	Type of the verification run.
<code>veri_run_class</code>	byte	Table 30	Class of the verification run.
<code>veri_initial_date</code>	char(12)	yyyymmddhhmm	Start time of the 'model' run. specifically: forecast: start time of the forecast; 3D-Var and LETKF analysis: analysis time; nudging analysis: start time of the nudging run; first guess: start time of the forecast run used as the first guess at <code>verification_ref_time</code> .
<code>veri_forecast_time</code>	int	hhhhmm	Forecast time at <code>verification_ref_time</code> . Zero for all kinds of analyses.
<code>veri_resolution</code>	float(2)	degree	Model resolution, separately for the x- and y- dimension.
<code>veri_domain_size</code>	integer(3)		Domain size of the model. COSMO: nx, ny, nz GME: ni, nj, nz ICON: nxny, nxny, nz
<code>veri_description</code>	char(64)		Possibly more detailed description than run type: 'nudging run', 'etkf analysis', ...
<code>veri_ens_member</code>	int	Table 31	Ensemble member number with special meaning for non-positive numbers.
<code>veri_exp_id</code>	int		Experiment id of the verification run.

Table 5: NetCDF variables for verification data

3. Definition of Data Items

3.1. Observation and Code Types

Specification of the contents of the header variables obstype and codetype.

value	name	description	
1	SYNOP	SYNOP report,	(ECMWF convention)
2	AIREP	AIREP report,	(ECMWF convention)
3	SATOB	SATOB report (AMV),	(ECMWF convention)
4	DRIBU	DRIBU report,	(ECMWF convention)
5	TEMP	TEMP report,	(ECMWF convention)
6	PILOT	PILOT report,	(ECMWF convention)
7	SATEM	SATEM report,	(ECMWF convention)
8	PAOB	PAOB report.	(ECMWF convention)
9	SCATT	Scatterometer report,	(ECMWF convention)
10	RAD	Radiances	(ECMWF convention)
11	GPSRO	GPS Radio occultations,	(DWD convention)
12	GPSGB	GPS ground based observations	(DWD convention)
13	RADAR	RADAR (volume data)	(DWD convention)
14	POWER	POWER (win, solar) data	(DWD convention)
15	SOIL	Soil retrieval	(DWD convention)
16	OBJECT	Objects	(DWD convention)
17	LIGHTN	Lightning	(DWD convention)
18	WLIDAR	Atmospheric Lidar	(DWD convention)
19	MWR	Microwave radiometer	(DWD convention)

Table 6: Observation Types

value	name	description	
101	OCE_PROF	Ocean profile	(DWD convention)
102	OCE_TRAJ	Ocean trajectory	(DWD convention)
103	OCE_MOOR	Ocean moored report	(DWD convention)
104	OCE_SURF	Ocean surface report	(DWD convention)
105	OCE_SSH	Satellite sea surface height	(DWD convention)

Table 7: Oceanographic Observation Types

value	name	description	
11	SRSCD	synop surface report	

14	ATSCD	automatic synop surface report
20	CMAN	Coastal Marine Automated Network
19	CWS	Citizen Weather Station (Netatmo)
15	SWIS	Road weather station data (SWIS)
16	SSNOW	Snow observations from SYNOP
17	ALSD	Additional Land Surface Data
18	CARS	Car data
21	AHSCD	ship synop report
24	ATSHS	automatic ship synop report
140	METAR	METAR
110	GPS	GPS zenith delay
141	AIRCD	airep report
41	CODAR	codar report
144	AMDAR	amdar report
146	MODES	mode-s report
145	ACARS	acars report
147	TAMDAR	TAMDAR or AFIRS report
87	CLPRD	cloud (height) product
88	STBCD	satob report
90	AMV	AMV
165	DRBCD	dribu report
64	TESAC	scatterometer
35	LDTCD	land temp report
36	SHTCD	temp ship report
135	TDROP	temp-drop report
37	TMPMB	temp mobile
109	BTEMP	BUFR TEMP land (high resol.)
111	BSHIP	BUFR TEMP ship (high resol.)
230	BDROP	BUFR TEMP drop (high resol.)
231	TEMPD	BUFR TEMP descent (high resol.)
32	LDPCD	land pilot report
33	SHPCD	ship pilot report
38	PLTMB	pilot mobile
210	ATOVS	ATOVS satellite data (1dvar)
132	WP_EU	European wind profiler
133	RA_EU	European sodar/rass report
134	WP_JP	Japanese wind profiler
136	PR_US	wind/profiler/rass report (USA)
137	RAVAD	radar VAD wind profile report
139	TOWER	tower profile data
159	ICOS	tower profile data (ICOS)
157	SCADA	SCADA report
187	WL_GB	wind lidar (ground-based)
190	RAMAN	Raman lidar
191	DIAL	Differential absorption lidar
150	PWIND	wind power data
151	PWSOL	solar power data
218	SEVIR	SEVIRI
123	ASCAT	ASCAT scatterometer
122	QSCAT	QSCAT scatterometer
216	AIRS	AIRS
217	IASI	IASI
250	GPSRO	GPS Radio Occultation
251	GPSGB	GPS slant delay

305	ASCWS	ASCAT soil moisture retrieval
400	REFLOBJ	Reflectivity object
401	STATIST	Statistical object
201	GBLIGHT	Ground-based lightning data
811	SYNTST	test synop surface report
835	SYTEMP	TEMP-derived surface report
839	SYTOWR	tower-derived surface report
501	MWR	MW radiom. data (ground-based)
850	SRSCD_ROF	SYNOP surface (rof)
851	CWS_ROF	SYNOP CWS (rof)
852	LDPCD_ROF	land PILOT (rof)
853	IAGOS_ROF	airep report (rof)
860	AOD_ROF	aerosol optical depth (rof)
854	EDP_ROF	eddy diss. parameter (rof)
600	OCE_OSTIA_SST	OSTIA SST product
601	OCE_SSH	SSH from satellite
602	OCE_SMOS_L2	Level 2 SMOS product
603	OCE_SMOS_L3	Level 3 SMOS product
604	OCE_SEAICE	sea ice product
605	OCE_SWH	Significant Wave Height from satellite
650	OCE_ARGO	generic vertical ocean profile
651	OCE_GLIDER	ocean glider
652	OCE_ANIBOS	ocean obs from animal
653	OCE_MOOREDDB	ocean obs from moored buoy
654	OCE_DRIBU	ocean obs from DRIBU
991	SYNOPDUM	dummy SYNOP
992	AIREPDUM	dummy AIREP
995	TEMPDUM	dummy TEMP
996	PILOTDUM	dummy PILOT

Table 8: Observation Code Types

3.2. Report and observation status

value	name	description
0	ACCEPTED	active and VQC accepted (used in 3D-Var only)
1	ACTIVE	used in the assimilation
3	MERGED	not used, merged into multilevel report
5	PASSIVE	not used, only monitored
7	REJECTED	not used due to suspicious quality
9	PAS_REJ	passive and rejected
11	OBS_ONLY	observation only, no model equivalent available
13	DISMISS	dismiss observation, should not appear in file

Table 9: Observation or report status values

Observations used in the assimilation are denoted as ACTIVE or ACCEPTED. The ACCEPTED flag indicates that the observation obtained a weight larger than 0.5 in the Variational Quality Control.

Observations not used in the assimilation are denoted as REJECTED if they are dismissed due to insufficient quality (did not pass all of the quality control checks). They are denoted as PASSIVE if they are not assimilated but processed by the assimilation system just for monitoring purposes. The PAS REJ flag indicates that passively monitored observations did not pass the quality control checks. The status MERGED refers to reports which were merged into others (Airep multilevel reports, TEMP parts A,B,C,D) and are thus redundant. The status OBS ONLY indicates that no model equivalent to the observation is present, either because the operator could not be applied (arguments were in an unphysical range, cf. flag OPERATOR) or because the information is only included for verification purposes.

3.3. Quality check flags

bit#	name	description	
2	SUSP_LOCT	suspicious location or date/time	1
3	TIME	time not in valid range	2
4	AREA	location not in valid area	3
8	PRACTICE	bad reporting practice/insuff. data	4
9	DATASET	dataset quality flags	5
1	BLACKLIST	blacklist (or not whitelist)	6
5	HEIGHT	location not in valid height range	7
6	SURF	incorrect surface (land,ice,etc)	8
7	CLOUD	cloud check	9
16	GROSS	gross error flag	10
0	OBSTYPE	passive report type (at obs.location)	11
10	REDUNDANT	redundant report	12
11	FLIGHTTRACK	flight track error flag	13
12	MERGE	merged reports (e.g. TEMP ABCD)	14
13	THIN	thinning	15
14	RULE	complex rule	16
17	NO_BIASCOR	no bias correction available	17
15	OBS_ERR	observation error too large	18
19	NO_OBS	no observations in report	19
18	FG	observation - first guess check	20
21	FG_LB	obs- lateral boundary condition check	21
20	OPERATOR	observation operator not applicable	22
32	NONE	no flag set	

Table 10: Report or Observation Flags

If the report or observation flag value is OBSTYPE or THIN, the status flag will be set to PASSIVE. For a value of MERGE the status becomes MERGED. Other report or observation flag values will lead to a status of REJECTED.

The entries of table 10 are used as bit numbers (using Fortran convention, 0 being the least significant bit). If the observation has not passed a check the respective bit is raised in variables flags or r_flags. Variables

check and r_check hold the number of the check that first caused the rejection of the data. In different assimilation systems the checks may be applied in different orders. The sequence number (right column in table 10 corresponds to the order used within COSMO. It may be used to derive a reasonable value of variable check from flags if not given otherwise.

The value NONE is only used in variables check and r_check to indicate that no bit was set in flags or r_flags.

3.4. Observation variable and level types

value	name	units	description	
0	NUM		ordinal (channel) number	L
3	U	m/s	u-component of wind	
4	V	m/s	v-component of wind	
8	W	m/s	vertical velocity	
1	Z	gpm	geopotential height	(also L)
57	DZ	(m/s)**2	thickness	
9	PWC	kg/m**2	precipitable water content	
28	TRH	0..1	transformed relative humidity	
29	RH	0..1	relative humidity	
58	RH2M	0..1	2 metre relative humidity	
17	PRH	0..1	pseudo relative humidity	
46	PRH2M	0..1	2 metre pseudo relative humidity	
10	LWC	kg/m**2	liquid water content	
2	T	K	upper air temperature	
59	TD	K	upper air dew point	
39	T2M	K	2 metre temperature	
40	TD2M	K	2 metre dew point	
11	TS	K	surface temperature	
12	TSEA	K	sea/water temperature	
30	PTEND	Pa/3h	pressure tendency	
60	W1	WMO 020004	past weather	
61	WW	WMO 020003	present weather	
62	VV	m	visibility	
63	CH	WMO 020012	type of high clouds	
64	CM	WMO 020012	type of middle clouds	
65	CL	WMO 020012	type of low clouds	
66	NH	m	cloud base height (cover >= 1/8)	
96	CEIL	m	cloud ceiling AGL (cover >= 5/8)	
67	N_L	WMO 020011	low cloud amount	
93	N_M	WMO 020011	medium cloud amount	
94	N_H	WMO 020011	high cloud amount	
69	C	WMO 500	additional cloud group type	L
70	NS	WMO 2700	additional cloud group amount	
71	SDEPTH	m	snow depth	
72	E	WMO 020062	state of ground	
79	TRTR	h	time period of information	L
80	RR	kg/m**2	precipitation amount	
81	TMAX	K	maximum temperature	
87	GCLG	Table 6	general cloud group	
91	N	WMO 020011	total cloud amount	

92	SFALL	m	6h snow fall	
110	PS	Pa	surface (station) pressure	
111	DD	degree	wind direction	
112	FF	m/s	wind speed	
118	REFL	0..1	reflectance	
119	RAWBT	K	brightness temperature	
120	RADIANCE	W/sr/m**3	radiance	
41	U10M	m/s	10m u-component of wind	
42	V10M	m/s	10m v-component of wind	
7	Q	kg/kg	specific humidity	
45	Q2M	kg/kg	2 metre specific humidity	
56	VT	K	virtual temperature	
154	DEPTH	m	depth below surface	
155	CTH	m	cloud top height	
156	HEIGHT	m	height	L
153	HOSAG	m	height of sensor above ground	L
157	FLEV	m	nominal flight level	L
158	ELEV	degree	elevation	L
179	NSOILM	0..1	normalized soil moisture	
180	SOILM	m**3/m**3	volumetric soil moisture	
226	MIXR	kg/kg	water vapour mixing ratio	
230	PWIND	W	wind power data	
231	PWSOL	W	solar power data	
192	RREFL	dB	radar reflectivity	
193	RADVEL	m/s	radial velocity	
194	HLOS	m/s	horizontal line of sight wind	
195	RZDR	dB	radar differential reflectivity	
196	RKDP	degree/m	radar specific differential phase	
197	RCC		radar correlation coefficient	
164	LWD	kg/m**3	liquid water density	
165	IWD	kg/m**3	ice water density	
166	NCL	1/kg	total number concentration: liquid	
167	NCI	1/kg	total number concentration: ice	
168	VMDL	m	volume mean diameter: liquid	
169	VMDI	m	volume mean diameter: ice	
128	PDELAY	m	atmospheric path delay	
162	BENDANG	rad	bending angle	
252	IMPPAR	m	impact parameter	L
248	REFR		refractivity	
245	ZPD		zenith path delay	
246	ZWD		zenith wet delay	
247	SPD		slant path delay	
240	VGUST	m/s	vertical gust (aircrafts)	
242	GUST	m/s	wind gust	
251	P	Pa	pressure	L
243	TMIN	K	minimum temperature	
236	RAD_DI	J/m**2	direct solar radiation	
237	RAD_GL	J/m**2	global solar radiation	
238	RAD_DF	J/m**2	diffuse solar radiation	
239	RAD_LW	J/m**2	long-wave (downward) radiation	
241	PRED	Pa	reduced pressure	
244	TURB	WMO 011031	degree of turbulence	
228	TURBIND	WMO 011037	turbulence index	
249	NFXME	m/s	max wind speed (10min mean)	

95	ICLG	Table 7	individual cloud layer group
174	AOD		aerosol optical depth
186	CO2	kg/kg	carbon dioxide
188	CH4	kg/kg	methane
189	N2O	kg/kg	nitrous oxide
253	FR_ICE	0..1	sea ice area fraction
220	SWH	m	significant wave height
224	SALINITY	PSU	ocean practical salinity
254	SWPT	K	sea water potential temperature
255	SWT	K	sea water temperature
273	SSH	m	sea surface height
287	SLA	m	sea level anomaly
500	OBJ_LAT	degree	centroid latitude
501	OBJ_LON	degree	centroid longitude
502	OBJ_Z	m	centroid height
503	OBJ_AREA	m**2	area of projected polygon
504	OBJ_CVIL	kg/m**2	cell-based vertical integrated liq.
505	OBJ_NUM		number of objects
600	LIGH_FLR	/m**2/s	flash rate
991	DUMMY1		dummy varno for preliminary use
992	DUMMY2		dummy varno for preliminary use
993	DUMMY3		dummy varno for preliminary use

Table 11: Observation and level variable numbers

3.5. General and individual cloud group

In addition to the entries in the table below, level significance is encoded in the variable level_sig (WMO table 8002, 6 bits)

value	name	units	description
0	CLBP	WMO 020011	bit position for cloud amount
4	LCBP	WMO 020012	bit position for low cloud type
10	MCBP	WMO 020012	bit position for middle cloud type
16	HCBP	WMO 020012	bit position for high type
4	CLOC	WMO 020011	no.bits used for cloud amount
6	LCOC	WMO 020012	no.bits used for low cloud type
6	MCOC	WMO 020012	no.bits used for middle cloud type
6	HCOC	WMO 020012	no.bits used for high type

Table 12: general cloud group bit positions

value	name	units	description
-------	------	-------	-------------

0	CLBP	WMO 020011	bit position for cloud amount
4	CTBP	WMO 020012	bit position for cloud type
10	BSBP	m	bit position for cloud base height
4	CLOC	WMO 020011	no.bits used for cloud amount
6	CTOC	WMO 020012	no.bits used for cloud type
14	BSOC	m	no.bits used for cloud base height

Table 13: individual cloud group bit positions

3.6. Level significance

bit#	name	description
0	SURFACE	surface
1	STANDARD	standard level
2	TROPO	tropopause level
3	MAX	maximum wind level
4	SIGN	significant level
5	SUPEROBS	superobservation (layer average)

Table 14: level significance for TEMP/PILOT.

For general and individual cloud group level significance is encoded according to WMO table 8002.

3.7. Specification of Instruments and computational methods

value	name	description
17	GRAW	Graw (D)
26	BASORA	Basora (CH)
27	RS_RU_A_MRZ	AVK-MRZ (Russia)
28	RS_RU_MET1	Meteorit Marz2-1 (Russia)
37	RS_80	Vaisala RS 80
49	RS_VIZ_M2	VIZ MARK II (USA)
57	RS_DC_MODEM	M2K2-DC Modem (France)
58	RS_RU_A_BAR	AVK-BAR (Russia)
71	RS_90_DIG12	Vaisala RS 90 Digicora I,II or Marvin
75	RS_RU_ARMA	AVK-MRZ-ARMA (Russia)
79	RS_92_DIG12	Vaisala RS 92 Digicora I,II or Marvin
80	RS_92_DIG3	Vaisala RS 92 Digicora III
81	RS_92_AUTO	Vaisala RS 92 Autosonde

88	RS_RU_V_MRZ	MARL-A or Vektor-M-MRZ (Russia)
89	RS_RU_V_BAR	MARL-A or Vektor-M-BAR (Russia)
99	RS_SA_DAT4G	BAT-4G (South Africa)
255	MISS	missing value

Table 15: radiosonde type (NRARA), for other values see WMO common code table C2

value	name	description
0	NOWIND	no windfinding
2	AUX_OPTIC	automatic with aux. optical direction finding
3	AUX_RANGE	automatic with auxiliary ranging
6	LORANC	automatic cross chain Loran-C
8	SATNAV	automatic satellite navigation
19	NOTSPEC	tracking technique not specified
70	NORMAL	all systems in normal operation
127	MISS	missing value
18	AIR_PHASE	aircraft obs: flight phase from data assimil.

Table 16: tracking technique (NSASA), for other values see WMO common code table C7

value	name	description
1	IR	infrared channel
2	VIS	visible channel
3	WV	water vapour channel
101	IR1	8.7 um Meteosat 8-11, 10.7 um GOES 10-17
201	IR2	9.7 um Meteosat 8-11, 3.9 um GOES 10-17
301	IR3	10.8 um Meteosat 8-11
102	VIS1	0.6 um Meteosat 8-11, 0.65um GOES 10-17, 0.65um Himawari 8-9
202	VIS3	0.8 um Meteosat 8-11
302	VIS2-HR	0.75um Meteosat 8-11
103	WV1	6.2 um Meteosat 8-11, 7.4 um GOES 10-17, 6.25um Himawari 8-9
203	WV2	7.3 um Meteosat 8-11, 7.0 um GOES 10-17, 6.95um Himawari 8-9
303	WV3	6.8-6.2 um GOES 10-17, 7.35um Himawari 8-9

Table 17: satellite derived wind computation method

value	name	description
0	PRESS	pressure instrument associated with wind measuring equipment

1	OPTTHEO	optical theodolite
2	RADTHEO	radio theodolite
3	RADAR	radar
4	VLFOMEGA	VLF-Omega
6	WINDPROF	wind profiler
5	LORANC	Loran-C
7	SATNAV	satellite navigation
8	RASS	radio acoustic sounding system (RASS)
9	SODAR	SODAR
10	LIDAR	LIDAR
15	MISS	missing

Table 18: type of measuring equipment used (NA4, 002003)

value	name	description
0	NO	no correction
1	CS_CI	CIMO solar + CIMO infrared corrected
2	CS_IN	CIMO solar + infrared corrected
3	CS	CIMO solar corrected only
4	SO_IN_AUTO	solar + infrared corr., automatic. by rsond. system
5	SO_AUTO	solar corrected automatically by radiosonde system
6	SO_IN_CNTRY	solar + infrared corr. as specified by country
7	SO_CNTRY	solar corrected by country
15	MISS	missing

Table 19: solar and infrared radiation correction (NSR, 002013)

3.8. Observation operator specific settings

value	name	description
0	MISSING	missing value
1	RAD_CLEAR_SKY	clear sky radiances
2	RAD_CLOUDY	cloudy radiances
3	BT_CLEAR_SKY	clear sky br.temp.

Table 20: Observation operator flag (observation type dependent)

3.9. 1DVAR Surface type, cloud flags and processing flags

bit#	name	description
0	LAND	land
1	SEA	sea
2	ICE	seaice
3	NOICE	ice-free sea
4	SNOW	snow
5	NOSNOW	snow-free land
6	HIGHLAND	highland
7	MWSURF	bad surface type detected by MW sounder
8	BLK_PP	surface blacklisted by sat_pp
9	MISMATCH	mismatch of surface types
10	MISSING	no surftype available

Table 21: surface types consistent with 1d-Var

bit#	name	description
0	CLEAR	
1	IR_CLOUDY	
2	MW_CLEAR	
3	MW_CLOUDY	

Table 22: cloud flag

value	name	description
1	EMIS_SEA_MOD	emiss. calc. with sea mod. (FASTEM,ISEM,...)
1	EMIS_DYNRET	emiss. calc. with dynamical retrieval
2	EMIS_2	emiss. from Grody(MW), CAMEL2007(IR)
3	EMIS_3	emiss. from TELSEM(MW), CAMEL clim.(IR) atlas
4	EMIS_4	emiss. from CNRM(MW), UWIR(IR) atlas
5	EMIS_5	emiss. from BRDF(VIS) or other atlas
6	EMIS_PC_RETR	emiss. from PC retrieval
7	EMIS_FAILED	emiss./reflectance calc. failed
8	SURF_INFL	surface influence too large
9	SURF_RETR	wrong surface type retrieved
10	SURF_TYPE	wrong surface type (not activated in namelist)
11	SURF_MODEL	surface properties (given by model) not adequate
12	CLOUD1	cloud check 1
13	CLOUD2	cloud check 2
14	CLOUD3	cloud check 3

15	CLOUD4	cloud check 4
16	CLOUD5	cloud check 5
17	CLOUD6	cloud check 6
18	AEROSOL	aerosol affected
19	DESERT_DUST	desert dust affected
20	VOLCANIC_ASH	volcanic ash affected
21	TRACE_GAS	trace gas affected
23	TSKIN_DYNRET	skin temperature calc. with dynamical retrieval
24	TSKIN_FAILED	skin temperature calc. failed
22	BC_FAILED	biascorrection failed

Table 23: TOVS specific flag

value	name	description
0	NOPR	non-processed containing no data or corrupted data
1	LAND_FREE	cloud free land
2	SEA_FREE	cloud free sea
3	LAND_SNOW	land contaminated by snow
4	SEA_ICE	sea contaminated by snow/ice
5	CUM_VLOW	very low and cumuliform clouds
6	STRAT_VLOW	very low and stratiform clouds
7	LOW_CUM	low and cumuliform clouds
8	LOW_STRAT	low and stratiform clouds
9	MED_CUM	medium and cumuliform clouds
10	MED_STRAT	medium and stratiform clouds
11	HI_OP_CUM	high opaque and cumuliform clouds
12	HI_OP_STRAT	high opaque and stratiform clouds
13	VH_OP_CUM	very high opaque and cumuliform clouds
14	VH_OP_STRAT	very high opaque and stratiform clouds
15	HL_ST_THIN	high semitransparent thin clouds
16	HL_ST_MEAN	high semitransparent meanly thick clouds
17	HL_ST_THICK	high semitransparent thick clouds
18	HL_ST_ABOVE	high semitransparent above low or medium clouds
19	FRAC	fractional clouds (sub-pixel water clouds)
20	UNDEF	undefined (undefined by CMa)

Table 24: Cloud type according to NWC SAF

3.10. Model surface characteristics

bit#	name	description
0	LAND	set if some fraction is covered by land

1	SEA	set if some fraction is covered by sea
2	ICE	set if some fraction is covered by sea-ice
3	NO_ICE	set if some fraction is not covered by sea-ice
4	SNOW	set if some fraction is covered by snow
5	NO_SNOW	set if some fraction is not covered by snow

Table 25: model surface characteristics

Values are missing if corresponding bits (eg. LAND and SEA or ICE and NO ICE) are set to zero.

3.11. Soil types

value	name	description
1	ICE	
2	ROCK	
3	SAND	
4	SANDLOAM	
5	LOAM	
6	CLAYLOAM	
7	CLAY	
8	PEAT	
9	SEAWATER	
10	SEAICE	

Table 26: type of soil in the retrieval point

3.12. Observation quality information

value	name	description
2	UNS	Unsteady
3	LVR	Level flight, routine observation
4	LVW	Level flight, highest wind encountered
5	ASC	Ascending
6	DES	Descending
7	MIS	Missing

Table 27: aircraft phase, radiances fov, gpsro pcd

value	name	description
0	GOOD	Good
1	BAD	Bad
3	MIS	Missing value

Table 28: aircraft roll angle

3.13. Specification of verification runs

value	name	description
0	FORECAST	forecast
1	FIRSTGUESS	first guess
2	PREL_ANA	preliminary analysis in observation space
3	ANALYSIS	analysis
4	INIT_ANA	initialised analysis
5	LIN_ANA	linear operator on analysis (Y _a)
6	FC_SENS	forecast sensitivity

Table 29: type of verification run

value	name	description
0	HAUPT	main forecast cycle
1	VOR	pre-run
2	ASS	assimilation cycle
3	TEST	test (offline)

Table 30: class of verification run

value	name	description
0	ENS_MEAN	ensemble mean
-1	DETERM	deterministic model run
-2	ENS_SPREAD	ensemble spread
-3	BG_ERROR	3dvar background error
-4	TALAGRAND	Talagrand index

-5	VQC_WEIGHT	variational quality control weight
-6	MEMBER	generic value for ensemble member
-7	ENS_MEAN_OBS	ensemble mean in observation space
-8	BIASCOR	bias correction

Table 31: specification of the verification data

The value MEMBER is not actually written to the file. It is merely used to specify all ensemble members generically in certain subroutine calls.

ENS_MEAN_OBS indicates the ensemble mean taken in observation space, in contrast to ENS_MEAN which is derived by application of the observation operator to the ensemble mean in model space.

4. Migration of COSMO VOF format to NetCDF format

4.1. File Header

The information in the VOF file header is represented in the NetCDF file as follows:

1) Verification period:

'initial date and hour' renamed to 'verification_reference_time'
(yyyymmddhhmm), now in general 'end of assimilation period'

'start', 'end' renamed to 'verification_start', 'verification_end'
(minutes)

2) LM-grid: pole, corners: 'lat_pole', 'lon_pole',
 'lat_lower_left', 'lon_lower_left'
 'lat_upper_right', 'lon_upper_right'

resolution, domain size: to be generalised for GME GRID

3) initial time and date : not used
QC time step : not used
QC thresholds : not used

4) Number of model runs to compare with observations: -> 'n_veri'

This quantity becomes a dimension (not a global attribute)

5) Further information on the verification runs:

The following quantities are stored as data (variables of dimension 'n_veri'), not as global attributes.

Domain used for verification : not used
 Types assigned for model runs : -> 'run_type' to be redefined:
 0 forecast
 1 first_guess
 2 preliminary_analysis
 3 analysis
 4 initialised_analysis

'run_type'
 'initial_date' (yyyymmddhhmm)
 'forecast_time' (hhmm) at verification_reference_time
 'mesh_width' (float) degree
 'n_levels'
 'description' possibly more detailed than 'run_type'
 'nudging run', etkf analysis', ...

additionally:

'ens_member' ensemble member number, special
 meaning for non-positive values:
 0 : ensemble mean
 -1 : deterministic run
 -2 : ensemble spread
 -3 : bg_error (3DVAR)

 'exp_id' experiment id

4.2. Report Header

The information in the VOF file is represented in the NetCDF file as follows:

- 1) basic report type : not used
- 2) station identity : 'statid' character (len_010)
- 3) longitude : 'lon' float (degree)
- 4) latitude : 'lat' float (degree)
- 5) observation time : 'time' int observation - verification_reference_time
in minutes
- 6) station altitude : 'z_station' short (m)
- 7) model orography : 'z_model' short (m)
- 8) observation type : 'obstype' short
 - 1 SYNOP
 - 2 AIREP
 - 3 SATOB (AMV)

			4 DRIBU
			5 TEMP
			6 PILOT
			7 SATEM (ATOVS)
			8 PAOB
			9 SCATT
			10 GPS RO
			11 GPS ground based (was =8 in VOF)
9) observ. code type :	'codetype'	short	cf. Figure 5.10, revised (see Table 5)
10) station characteristics	: split up in different variables, bit positions		
	in VOF:		
0,1	'r_state'	byte	merged, passive, rejected, active
2-6,9,20,21	'r_flags'	int	bit pattern of flags: station location, height distance, blacklist, code type excluded, redundant, suspicion indicator (--> DATASET)
	'r_check'	byte	check which caused 'r_state' (one out of 'r_flags')
22-29	'phase'	byte	aircraft phase and roll angle (bit-Feld, mit INFO in BUFR abgleichen)
13-19	'instype'	int	station type or satellite instrument
10	cancelled		(important station indicator)
	7) sea grid point		
	8) station correction indicator: adopt		
11) report flags on lat/lon/date/time/altitude bit pattern, align with INFO in BUFR			
	'r_flags'	int	bit pattern of flags: station location,
12) status 0/1/2: active/single_level_aircraft_set_passive/passive			
	'r_state'	byte	merged, passive, rejected, active
13) threshold quality control (QC) for extrapolated surface pressure			
	'flags'	int	in body: bit pattern of flags: fg-check etc.
14) model index x	'index_x'	int	
15) model index y	'index_y'	int	
	model index diamond 'index_d'	int	

5. Fortran 90 interface

The Fortran 90 modules **mo_t_table**, **mo_fdbk_tables**, **mo_t_netcdf_file**, **mo_fdbk**, **mo_fdbk_io**, and **test_feedback** are intended for common use in the COSMO model, the DWD 3D-Var and the LETKF.

The modules **mo_t_table** and **mo_t_netcdf_file** hold general data type definitions and operations for NetCDF file handling, not specific to the feedback-file layout.

Modules **mo_fdbk_tables** and **mo_fdbk** hold variable definitions (table entries) and routines specific the the feedback file. These modules may be used for low level I/O routines which access the feedback file content directly (variable by variable).

Module **mo_fdbk_io** provides higher level routines to read/write the feedback file and derived types to hold the complete file content in memory. Thus application of these routines puts higher demands on memory usage but is easier to use may be more appropriate if no direct storage into model or assimilation system specific data structures is intended. Module **mo_fdbk_rad** builds up on **mo_fdbk_io** and further facilitates writing of feedback files for radiance data.

The module **mo_fdbk_3dvar** holds routines to write the feedback file. This module is specific for the DWD 3D-Var and is provided only as an example. This example is much more complex than **mo_fdbk_io** or **mo_fdbk_rad** as it involves MPI parallelisation and storage in the 3D-Var specific data structures.

The program **test_feedback** writes an (empty) feedback file and prepares the LaTeX sources (tab....tex) with the table entries used in this document. It may serve as an example program how to write feedback files.

The program **test_feedback_read** reads a feedback file either via the low level routines from **mo_fdbk** or via the higher level routines from **mo_fdbk_io** and does some calculation and printout. It may serve as an example program how to read feedback files.

The program **test_feedback_rad** is an example routine which writes a feedback file for radiance data using modules **mo_fdbk_io** and **mo_fdbk_rad**.

5.1. Tables

5.1.1. Data type definitions (mo_t_table)

Derived data types and operators to maintain tables with the description of the data content (as defined in Section 3) are defined within this module.

Derived type definitions:

t_entry

Derived type to hold a table entry relating the value of a datum with a name (mnemonic), a description, and units (if applicable):

```
integer, parameter :: NLEN = 16 ! length of entry name
integer, parameter :: ULEN = 12 ! length of units field
integer, parameter :: DLEN = 64 ! length of entry description

type t_entry
  integer          :: value          ! numerical value or bit number
  character(len=NLEN) :: name        ! associated name (mnemonic)
  character(len=ULEN)  :: units       ! units of the numerical value
  character(len=DLEN)  :: description ! some text
end type t_entry
```

t_table

Derived type to hold a table:

```

type t_table
  type (t_entry) ,pointer :: e(:)      ! list of table entries
  character(len=16)      :: name      ! name of table
  character(len=128)     :: caption ! caption (for LaTeX doc)
  integer                :: n         ! number of entries
  integer                :: first     ! smallest value or bit number in table
  integer                :: last      ! largest value or bit number in table
  logical                :: bit       ! used as bit-table
end type t_table

```

The flag `bit` indicates if the values are bit numbers used in a flag table.

Subroutines and functions:

init_table

```

subroutine init_table (table, entries, name, caption, bit, latex)
type (t_table)          ,pointer :: table      ! table
type (t_entry)          ,intent(in) ,target  :: entries(:) ! table entries
character(len=*)         ,intent(in)          :: name      ! name of table
character(len=*)         ,intent(in)          :: caption    ! caption (for LaTeX)
logical                 ,intent(in)          :: bit         ! used as bit flag
logical                 ,intent(in) ,optional :: latex      ! write LaTeX file

```

Subroutine to set the values of a table:

1. allocate the pointer `table`.
2. link the pointer component `table% e` with the table entries.
3. derive the components `n` (number of entries), `first`, and `last` (smallest and largest value in the table).
4. set `name`, `caption` and `bit` components with the values of the actual arguments.
5. if `latex` is passed with the value `.true.` write a latex file `tab.name.tex`. This option was used to prepare the tables within Section 3. In the LaTeX sources the following sequences are replaced:


```

      ' _ ' by '\_ '
      ' % ' by '\% '
      ' # ' by '\# '
      ' ~ ' by '\hfill '
      
```

name_value

```

elemental function name_value (table, value) result (name)
type (t_table)          ,intent(in) :: table
integer                 ,intent(in) :: value
character(len=NLEN)     :: name

```

determines the name of a table entry from its value or bit number.

value_name

```

    elemental function value_name (table, name) result (value)
    type (t_table)    ,intent(in) :: table
    character(len=*) ,intent(in) :: name
    integer           :: value

```

determines the value or bit number of a table entry from its name. If a table entry cannot be found a value of `INVALID_VALUE=-999` is returned.

position

```

    elemental function position (table, name)
    type (t_table)    ,intent(in) :: table
    character(len=*) ,intent(in) :: name
    integer           :: position

```

or

```

    elemental function position (table, value)
    type (t_table) ,intent(in) :: table
    integer        ,intent(in) :: value
    integer        :: position

```

determines the position of the entry in the table from its name or value.

5.1.2. Feedback file tables (`mo_fdbk_tables`)

In this module the tables (variables of derived type `t_table`) are defined. The module has the following public entities:

The tables used in Section 3 of this document:

```

type (t_table) :: status, flags, obstype, codetype, varno, runtype, runclass,
satsens, rsondtype, trackteqn, meas_equip, radiation_corr, surftype, flg_idvar,
flg_cld, level_sig, phase, rollangle, retrtype, ensmem

```

Constant definitions for the table entry values:

```

integer, parameter :: ST_ACCEPTED, ST_ACTIVE, .. for the entries of table status.
integer, parameter :: FL_OBSTYPE, FL_BLACKLIST, .. for the entries of table flags.
...

```

`init_fdbk_tables` (subroutine)

```

subroutine init_fdbk_tables (latex)
logical, intent(in), optional :: latex

```

Subroutine to initialise the tables (variables of derived type `t_table`) defined in this module. join table entries and table meta data (name and caption), optionally (if 'latex' is given and true) write LaTeX file with tables for inclusion in the documentation.

Actually subroutine `init_table` defined in module `mo_t_table` is called for each table with suitable parameters.

5.2. NetCDF interface

5.2.1. Data type definitions (`mo_t_netcdf_file`)

This module defines the derived type `t_netcdf_file` which mirrors some of the meta data of a NetCDF- file. The subroutines `add_dim` and `add_var` add a NetCDF-dimension or a NetCDF-variable to a variable of

this type. Subroutine **create_netcdf_file** creates (writes) an empty (dimension and variable definitions) NetCDF file based on the information in the variable. Subroutines **open_netcdf_file_read** and **open_netcdf_file_write** open a netcdf file for read or write access, respectively. Subroutines to actually write the data (contents of the variables) are not yet implemented.) Subroutine **close_netcdf_file** closes the NetCDF file. Subroutine **destruct_netcdf_file** finally deallocates pointer components of a variable of type **t_netcdf_file** when it is not used any more.

Derived type definitions:

t_netcdf_file

Derived type to hold some of the meta-data of a NetCDF-file:

```
integer, parameter :: NLEN = 32    ! len of variables name
integer, parameter :: LLEN = 64    ! len of longname
integer, parameter :: ULEN = 16    ! len of units attribute
integer, parameter :: OLEN = 8     ! len of optionals string
integer, parameter :: ODIM = 4     ! size of optionals array
integer, parameter :: PLEN = 128   ! len of file path/name
integer, parameter :: MDIM = 4     ! max. number of dimensions

type t_netcdf_file
  character(len=PLEN)      :: path      = ''      ! file path/name
  integer                  :: status    = UNDEFINED ! file status
  integer                  :: error     = NF_NOERR  ! error return value
  integer                  :: ndim      = 0         ! number of dimensions
  integer                  :: nvar      = 0         ! number of variables
  integer                  :: ncid      = 0         ! NetCDF file id
  type (t_netcdf_dim), pointer :: dims (: ) => NULL() ! dimensions
  type (t_netcdf_var), pointer :: vars (: ) => NULL() ! variables
end type t_netcdf_file
```

The pointer component **dims** holds information on the NetCDF-dimensions:

```
type t_netcdf_dim
  character(len=NLEN) :: name      = ''      ! name of dimension
  integer              :: len      = -1      ! length of dimension
  logical              :: unlimited = .false. ! unlimited dimension ?
  integer              :: dimid     = -1      ! NetCDF dimension id
  integer              :: pos       = 0       ! index of this entry in array
end type t_netcdf_dim
```

The pointer component **vars** holds information on the NetCDF-variables:

```
type t_netcdf_var
  character(len=NLEN) :: name      = ''      ! name of the variable
  character(len=LLEN) :: longname  = ''      ! CF convention
  character(len=ULEN) :: units     = ''      ! CF convention
  type(t_table), pointer :: table  => NULL() ! table of valid values
  integer              :: invalid  = -huge(1) ! invalid value for ..
  real                 :: rinvalid = -huge(1.) ! .. this variable
  integer              :: nvdims   = 0        ! number of dimensions
  type(p_netcdf_dim)   :: p (MDIM)           ! pointer to dimensions
  character(len=OLEN)  :: optionals(ODIM) = '' ! mark optional variables
  logical              :: opt_used = .false.  ! indicate used opt.variable
  integer              :: xtype    = -1       ! NetCDF data type
  integer              :: varid    = -1       ! NetCDF variable id
end type t_netcdf_var
```

The array component **p** holds pointers to the NetCDF dimensions of the variable:

```
type p_netcdf_dim
  type(t_netcdf_dim) ,pointer :: dim => NULL()
end type p_netcdf_dim
```

Subroutines:

add_dim

```
subroutine add_dim (file, name, len, pos, unlimited)
type (t_netcdf_file) ,intent(inout)      :: file      ! NetCDF meta data
character(len=*)      ,intent(in)        :: name      ! name of dimension
integer               ,intent(in)        :: len       ! length of dimension
integer               ,intent(out)       :: pos       ! index in file% dims
logical               ,intent(in) ,optional :: unlimited ! unlimited dimension
```

Adds a dimension to the NetCDF file meta data.

add_var

```
subroutine add_var (file, name, xtype, pos, longname, units, table, opt, invalid, rinvalid)
type (t_netcdf_file) ,intent(inout)      :: file      ! NetCDF meta data
character(len=*)      ,intent(in)        :: name      ! name of the variable
integer               ,intent(in)        :: xtype     ! NetCDF data type
integer               ,intent(in)        :: pos (:)   ! pointer to dimensions
character(len=*)      ,intent(in)        :: longname  ! CF convention
character(len=*)      ,intent(in) ,optional :: units  ! CF convention
type(t_table)         ,pointer ,optional :: table    ! table to valid values
character(len=*)      ,intent(in) ,optional :: opt    ! optional flags
integer               ,intent(in) ,optional :: invalid ! fillvalue for ints
integer               ,intent(in) ,optional :: rinvalid ! fillvalue for reals
```

Add a variable to the NetCDF file meta data. The optional parameter **opt** may be passed with a list of mnemonics separated by blanks (for instance **TEMP SYNOP PILOT**) in order to indicate that the respective variables are optional. They will be actually written to the NetCDF file only if one of the mnemonics passed later as the actual argument **opt** of subroutine **create_netcdf_file** matches this string. The optional arguments **invalid** or **rinvalid** may specify a non-default fillvalue to be used for this variable.

create_netcdf_file

```
subroutine create_netcdf_file (file, cmode, opt)
type (t_netcdf_file)      ,intent(inout) :: file ! NetCDF meta data
integer                   ,optional ,intent(in) :: cmode ! NetCDF creation mode
character(len=*) ,optional ,intent(in) :: opt ! optional parameter flag
```

Create a NetCDF file from its meta data provided in variable file:

This routine creates a NetCDF file and writes its meta data. It only defines NetCDF-dimensions, NetCDF-variables and some variable attributes (CF conventions: longname, units). Global attributes and variable contents must be written separately. Variables are only defined if the optional parameter flags match the respective flags set by subroutine **add_var**.

open_netcdf_file_read

```
subroutine open_netcdf_file_read (file)
type (t_netcdf_file) ,intent(inout) :: file ! NetCDF meta data
```

Open the netCDF file for read access.

open_netcdf_file_write

```
subroutine open_netcdf_file_write (file)
  type (t_netcdf_file) ,intent(inout) :: file ! NetCDF meta data
```

Open the netCDF file for read/write access.

close_netcdf_file

```
subroutine close_netcdf_file (file)
  type (t_netcdf_file) ,intent(inout) :: file
```

Close a NetCDF file. This routine should be called after the NetCDF file was created by subroutine **create_netcdf_file** and global attributes and variable contents has been written.

destruct_netcdf_file

```
subroutine destruct_netcdf_file (file)
  type (t_netcdf_file) ,intent(inout) :: file
```

Clean up the NetCDF file meta data derived type variable: deallocate components. This routine should be called if the content of the variable **file** is not used any more.

5.2.2. Feedback file interface (mo_fdbk)

This module defines the derived type **t_fdbk**. In addition to the information on dimensions and variables (within component **nc** of derived type **t_netcdf_file**) the type holds feedback file specific information (number of data items actually written, global attributes) and meta data on the verification runs (within component **veri**. Subroutine **setup_fdbk** sets up the information within this data type, **create_fdbk** writes a NetCDF file based on the meta-data stored in a variable of this type.

Subroutines **open_fdbk_read** and **open_fdbk_write** open a feedback file for read or write access, respectively.

Subroutine **add_history** adds a history entry to the global attributes. **write_global_attributes** actually writes the global attributes to the file.

Subroutine **add_verification** adds verification meta data and writes it to the file.

Subroutine **read_meta** reads the attributes and verification meta data and **print_fdbk** produces a printout of this meta data.

Subroutine **get_varid** gets the NetCDF varid of a variable. **get_fillvalue** returns the fillvalue used for a specific variable. **write_fdbk_var** actually writes a variable to the file.

get_veri_index gets the index or indices of verification data items. **get_veri** reads and **write_veri** writes verification data.

Subroutine **close_fdbk** closes the NetCDF-file and **cleanup_fdbk** deallocates pointer components of **t_fdbk**.

All the routines are based on those defined in **mo_t_netcdf_file** but extend them by the feedback file specific parts.

Derived type definitions:

t_fdbk

```

type t_fdbk ! specific feedback file attributes
!-----
! global feedback file attributes
!-----
integer          :: n_hdr          = 0 ! number of header records used
integer          :: n_body         = 0 ! number of body records used
integer          :: n_veri         = 0 ! number of verification runs used
character(len=TLEN) :: title       = '' ! (CF) title
character(len=5)   :: version      = '' ! (CF) version.subversion
character(len=ILEN) :: institution = '' ! (CF) institution
character(len=HLEN) &              ! (CF) history
                  ,pointer :: history(:) =>NULL()
character(len=SLEN) :: source      = '' ! (CF) source
integer            :: refdate      = 0 ! reference date 'yyyymmdd'
integer            :: reftime      = 0 ! reference time 'hhmm'
integer            :: start        = 0 ! verification start (minutes)
integer            :: end          = 0 ! verification end (minutes)
real               :: resolution (2) = 0 ! resolution : lat,lon (degree)
integer            :: domain      (3) = 0 ! domain size: x,y,z
real               :: pole        (2) = 0 ! pole coordinates : lat,lon
real               :: lower_left (2) = 0 ! lower left corner of domain
real               :: upper_right(2) = 0 ! upper right corner of domain
!-----
! generic NetCDF data structure
!-----
type (t_netcdf_file) :: nc          ! generic NetCDF data structure
!-----
! verification run meta data
!-----
type(t_fdbk_meta) ,pointer :: veri(:) =>NULL() ! verification run meta data
end type t_fdbk

```

Stores NetCDF meta data (global attributes) of a feedback file.

t_fdbk_meta

```

type t_fdbk_meta
character(len=MLEN) :: model        = ' ' ! model used for verification
integer            :: run_type      = -1 ! type of model run
integer            :: run_class     = -1 ! class of model run
character(len=IDLEN) :: initial_date = ' ' ! start of verification period
integer            :: forecast_time = 0 ! forecast time at verification_ref_time
real               :: resolution (2) = 0. ! model resolution (x,y)
integer            :: domain_size(3) = 0 ! domain size (x,y,z)
character(len=DLEN) :: description = ' ' ! detailed description
integer            :: ens_member    = 0 ! ensemble member number / special values
integer            :: exp_id        = -1 ! experiment Id
end type t_fdbk_meta

```

Stores meta data concerning the verification data.

Subroutines:

setup_fdbk

```

subroutine setup_fdbk (nc, latex)
type (t_netcdf_file) ,intent(out)          :: nc      ! NetCDF data type
logical               ,intent(in) ,optional :: latex ! write LaTeX tables

```

This subroutine sets up the information on dimensions and variables stored in variable **nc** of derived type **t_netcdf_file**. These information apply to feedback files in general but are not specific to a certain model (COSMO, GME) or report type (TEMP, SYNOP).

create_fdbk

```

subroutine create_fdbk (fb, path, model, version, institution, n_hdr,    &
                      n_body, reftime, reftime, start, end, resolution,&
                      domain, comment, time, runtime,                  &
                      pole, lower_left, upper_right, opt, create       )
!-----
! create new feedback file
! task 1: set up derived type according to actual parameters
! task 2: create file and write global attributes
!-----
type(t_fdbk)                ,intent(inout) :: fb          ! feedback file data type
character(len=*)            ,intent(in)   :: path         ! pathname
character(len=*)            ,intent(in)   :: model        ! model string
character(len=*)            ,intent(in)   :: version      ! model version
character(len=*)            ,intent(in)   :: institution  ! institution string
integer                     ,intent(in)   :: n_hdr        ! allocated size of header
integer                     ,intent(in)   :: n_body       ! allocated size of body
integer                     ,intent(in)   :: reftime      ! reference time yyyymmdd
integer                     ,intent(in)   :: reftime      ! reference time hhmm
integer                     ,intent(in)   :: start        ! verification start (minutes)
integer                     ,intent(in)   :: end          ! verification stop (minutes)
real                       ,intent(in)   :: resolution   (2)! model resolution (degree)
integer                     ,intent(in)   :: domain       (3)! domain size (x,y,z)
character(len=*)            ,intent(in)   :: comment      ! comment (for history)
character(len=*)            ,intent(in)   :: time         ! time (for history)
character(len=*) ,optional ,intent(in)   :: runtime      ! run time (for history)
real                       ,optional ,intent(in) :: pole   (2)! location of pole (degree)
real                       ,optional ,intent(in) :: lower_left (2)! lower left (lat,lon degree)
real                       ,optional ,intent(in) :: upper_right(2)! upper right (lat,lon degree)
character(len=*) ,optional ,intent(in)   :: opt          ! flag optional variables
logical                    ,optional ,intent(in) :: create ! if .false. postpone task 2

```

This subroutine stores information in variable **fb** of derived type **t_fdbk**. Information specific to the feedback file (global attributes, dimension sizes to allocate) are passed by actual arguments. Finally the file is actually created (unless parameter **create** is set to **.false.**).

open_fdbk_read

```

subroutine open_fdbk_read (fb, path)
type(t_fdbk)    ,intent(inout) :: fb  ! feedback file data type
character(len=*) ,intent(in)    :: path ! pathname

```

Opens a feedback file for read access.

open_fdbk_write

```

subroutine open_fdbk_write (fb, path)
type(t_fdbk)    ,intent(inout) :: fb  ! feedback file data type
character(len=*) ,intent(in)    :: path ! pathname

```

Opens a feedback file for write access.

add_history

```

subroutine add_history (fdbk, model, starttime, runtype, runtime, write)
!-----
! add history entry to global attributes
! 1) store in derived type
! 2) actually write to file
!-----
type (t_fdbk) ,intent(inout)      :: fdbk
character(len=*) ,intent(in)      :: model
character(len=*) ,intent(in)      :: starttime
character(len=*) ,intent(in)      :: runtype
character(len=*) ,intent(in) ,optional :: runtime ! (yyyymmddhhmm)
logical          ,intent(in) ,optional :: write   ! if .false. postpone 2)

```

Adds history entry to the global attributes (hold in the derived type) and writes them to the file (unless parameter **write** is set to **.false.**..

write_global_attributes

```

subroutine write_global_attributes (fb)
type (t_fdbk), intent(inout) :: fb

```

Write the global attributes hold in the derived type to the file.

add_verification

```

subroutine add_verification (fdbk, model, run_type, run_class, initial_date,&
                             fc_time, resolution, domain_size, description, &
                             ens_member, exp_id, id_veri, replace, ierr      )

```

Add an verification entry to the feedback file. All meta data entries (variables **veri.***) are written. Only the actual data (**veri.data**) has to be written in a subsequent step. If **replace** is present and **> 0**, the entry with the respective entry is replaced. Otherwise the entry is appended. The index actually used is returned in **replace**. By passing **ierr** a program abort due to an error condition may be captured. In this case **ierr** returns a value **/ = zero**.

read_meta

```

subroutine read_meta (fb)
type (t_fdbk), intent(inout) :: fb

```

Reads global attributes and verification meta data.

print_fdbk

```

subroutine print_fdbk (fdbk)
type (t_fdbk) ,intent(in) :: fdbk

```

Prints global attributes and verification meta data.

get_varid

```

function get_varid (fb, name) result (varid)
type (t_fdbk)      ,intent(in) :: fb    ! feedback file data type
character(len=*)   ,intent(in) :: name ! variable name
integer            :: varid ! NetCDF varid

```

Gets NetCDF varid of a variable in the feedback file. Returns -999 if the variable is not present.

get_fillvalue

```

function get_fillvalue (fb, name) result (fillvalue)
type(t_fdbk)      ,intent(in) :: fb      ! feedback file data type
character(len=*) ,intent(in) :: name    ! variable name
real              :: fillvalue ! fillvalue

```

Return the fillvalue used for a variable specified by its name.

write_fdbk_var

```

subroutine write_fdbk_var (fb, iv, var, fill)
type(t_fdbk)      ,intent(in) :: fb      ! feedback file meta data
integer           ,intent(in) :: iv      ! variable index
....              ,intent(in) :: var(:) ! variable to write
integer ,optional ,intent(in) :: fill    ! fillvalue to replace

```

Write a variable to the feedback file. `var` may be of type `integer`, `real(wp)` or `character(len=*)`. If `fill` is passed any element with this value will be replaced by the fillvalue used for the respective variable.

get_veri_index

```

subroutine get_veri_index (idx, nidx, fb, model, run_type, run_class,      &
                           initial_date, forecast_time, ens_member, exp_id)
integer           ,intent(out)      :: idx(:) ! indices returned
integer           ,intent(out)      :: nidx    ! number of indices
type (t_fdbk)     ,intent(in)       :: fb
character(len=*) ,intent(in) ,optional :: model
integer           ,intent(in) ,optional :: run_type
integer           ,intent(in) ,optional :: run_class
character(len=*) ,intent(in) ,optional :: initial_date
integer           ,intent(in) ,optional :: forecast_time
integer           ,intent(in) ,optional :: ens_member
integer           ,intent(in) ,optional :: exp_id

```

Returns the indices of verification entries specified by the (optional) arguments.

get_veri

```

function get_veri (fb, index) result (veri)
type (t_fdbk) ,intent(in) :: fb
integer       ,intent(in) :: index
real          :: veri(fb% n_body)

```

Actually reads a verification entry.

write_veri

```

type (t_fdbk) ,intent(in) :: fb
integer       ,intent(in) :: index
real(sp or dp)          :: veri(:)

```

Actually writes a verification entry.

close_fdbk

```

subroutine close_fdbk (fb)
type (t_fdbk) ,intent(inout) :: fb

```

Close a feedback file.

cleanup_fdbk

```

subroutine cleanup_fdbk (fb)
type (t_fdbk) ,intent(inout) :: fb

```

Deallocate components of a variable of derived type **t_fdbk** when not used any more.

Examples how to use this interface are provided in the subsequent Section 5.3

5.2.3. Feedback file I/O handling (mo_fdbk_io)

This module holds the derived type definition **t_fdbk_data** to hold the content of a feedback file and routines to read/write its content from/to a file. The subroutine **read_fdbk_data** reads the entries of a feedback file and stores them in a variable of type **t_fdbk_data**. Entries may be reorganised by deleting entries in the header and body components of the derived type variable by means of subroutine **pack_fdbk** or **associate_hb**. Subroutines **read_fdbk_veri**, **read_fdbk_var_head**, and **read_fdbk_var_body** may be used to read verification data and optional variables. Subroutine **write_fdbk_file** writes the content of the derived type variable. The content of a variable of type **t_fdbk_data** is deleted by subroutine **destruct**. The example program **test_feedback_read** (cf. section 5.3.2) shows how to use the routines from this module.

Derived type definitions:

t_fdbk_data

Container for feedback file data: header and body entries and verification meta data.

```

type t_fdbk_data
  type(t_fdbk)          :: f                ! feedback file meta data
  type(t_fdbk_head) ,pointer :: h (:)        => NULL() ! feedback file header
  type(t_fdbk_body) ,pointer :: b (:)        => NULL() ! feedback file body data
  real(wp)              ,pointer :: veri_data(:, :) => NULL() ! verification data
  type(t_fdbk_meta) ,pointer :: veri_meta (:) => NULL() ! verific.meta data
end type t_fdbk_data

```

t_fdbk_head

Feedback file header content

```

type t_fdbk_head
  !-----
  ! mandatory entries
  !-----
  integer      :: i_body      = ifill
  integer      :: l_body      = ifill
  integer      :: n_level     = ifill ! number of levels in body
  integer      :: obstype     = ifill
  integer      :: codetype    = ifill
  integer      :: center      = ifill ! station processing center
  integer      :: sub_center  = ifill ! processing subcenter
  integer      :: data_category = ifill
  integer      :: sub_category = ifill
  integer      :: ident       = ifill
  character(len=10) :: statid  = ''
  real(wp)      :: lat        = rfill
  real(wp)      :: lon        = rfill

```



```

integer      :: time          = ifill
integer      :: time_dbase    = ifill ! data base time
integer      :: time_nomi     = ifill ! nominal (synoptic) time
real(wp)     :: sun_zenit     = rfill ! solar zenith angle
integer      :: mdlslfc       = ifill
integer      :: z_station     = ifill ! station height
integer      :: z_modsurf     = ifill ! model surface height
integer      :: sta_corr      = ifill ! correction indicator
integer      :: instype       = ifill ! instrument type
integer      :: r_state       = ifill
integer      :: r_flags       = ifill
integer      :: r_check       = ifill
integer      :: index_x       = ifill
integer      :: index_y       = ifill
!-----
! some optional entries
!-----
integer      :: dbkz          = -1
integer      :: record        = -1
integer      :: subset        = -1
integer      :: source        = -1
integer      :: phase         = -1
integer      :: obs_id        = -1
integer      :: flg_1dvar     = -1
integer      :: flg_cld       = -1
integer      :: index_d       = -1
integer      :: surftype      = -1
real(wp)     :: sat_zenit     = -999._wp
!-----
! pointer to original position in file
!-----
integer      :: pos           = 0      ! position in file
!-----
! pointer to body entries
!-----
integer      :: ib            = 0      ! start index in body
integer      :: nb            = 0      ! number of body entries
end type t_fdbk_head

```

The default initialisation values correspond with the values the components will take for the case that an entry is not present in the file or that it takes its NetCDF fillvalue. They will be replaced by the NetCDF fillvalues when the file is written.

t_fdbk_body

Feedback file body content.

```

type t_fdbk_body
!-----
! content of feedback file
!-----
real(wp) :: obs      = rfill
real(wp) :: bcor     = rfill
real(wp) :: e_o      = rfill
real(wp) :: level    = rfill
real(wp) :: plevel   = -1._wp ! optional

```

```

integer :: level_typ = ifill
integer :: level_sig = ifill
integer :: varno      = ifill
integer :: state      = ifill
integer :: flags      = ifill
integer :: check      = ifill
integer :: qual       = ifill
!-----
! pointer to file
!-----
integer :: pos        = 0      ! position in file
!-----
! pointer to header entry
!-----
integer :: ih         = 0      ! index of header entry
end type t_fdbk_body

```

The default initialisation values correspond with the values the components will take for the case that an entry is not present in the file or that it takes its NetCDF fillvalue. They will be replaced by the NetCDF fillvalues when the file is written.

Subroutines:

read_fdbk_data

Reads entries of a feedback file and stores them in variable `data` of derived type `t_fdbk_data`.

```

subroutine read_fdbk_data (data, file)
type (t_fdbk_data) ,intent(inout) :: data
character(len=*)    ,intent(in)   :: file

```

pack_fdbk

```

subroutine pack_fdbk (fb, mask_h, mask_b)
type(t_fdbk_data) ,intent(inout) :: fb      ! feedback file data type
logical, optional ,intent(in)    :: mask_h (:) ! mask for header entries
logical, optional ,intent(in)    :: mask_b (:) ! mask for body entries

```

This routine applies some filtering on header and body entries stored in derived type variable `fb`. The header and body entries to keep are specified by the optional parameters `mask_h` and `mask_b`, respectively. The discarded entries are removed from `fb`. The references from header to body entries and vice versa are updated. Subsequent calls to `read_fdbk_var_head`, `read_fdbk_var_body`, and `read_fdbk_vari` only provide the reduced set of data.

associate_hb

```

subroutine associate_hb (data)
type (t_fdbk_data) ,intent(inout) :: data

```

Updates references from header to body entries and vice versa.

read_fdbk_var_head

```

subroutine read_fdbk_var_head (fb, name, var, ierr, fill)
type(t_fdbk_data) ,intent(in)  :: fb      ! feedback file data type
character(len=*)   ,intent(in)  :: name    ! variable name
return-type        ,intent(out) :: var (:) ! variable returned
integer ,optional  ,intent(out) :: ierr    ! error return parameter
integer ,optional  ,intent(in)  :: fill    ! fillvalue to use

```

Read a real or integer variable with name `name` from the header of the feedback file. The file must be read previously and stored in variable `fb`. The error return parameter is zero if the variable was read successfully. Fillvalues in the NetCDF file may be replaced by a different value provided by the optional parameter `fill`. `var` is either of type `integer` or `real(wp)`.

read_fdbk_var_body

```
subroutine read_fdbk_var_body (fb, name, var, ierr, fill)
  type(t_fdbk_data) ,intent(in)  :: fb      ! feedback file data type
  character(len=*)   ,intent(in)  :: name   ! variable name
  return-type        ,intent(out) :: var(:) ! variable returned
  integer ,optional ,intent(out) :: ierr    ! error return parameter
  integer ,optional ,intent(in)  :: fill    ! fillvalue to use
```

Same as `read_fdbk_var_head`, but reads a variable from the body of the feedback file.

read_fdbk_veri

Read the verification data and store it into `dataveri_data`. Verification data to be read is restricted by the optional parameters or by an explicit provision of the verification indices `ix`. The meta-data corresponding to the verification data read is provided in the component `data% meta_data`. It is a subset of the entries in `data% f% veri` selected according to the optional parameters. Observations to be read are restricted by a previous thinning and reorganisation of the derived type variable `data` (cf. subroutine `associate_hb`).

```
subroutine read_fdbk_veri (data, fill, ix, model, run_type, run_class, &
                          initial_date, forecast_time, ens_member, exp_id)
  type (t_fdbk_data) ,intent(inout)      :: data
  real(wp)           ,intent(in) ,optional :: fill    ! new fillvalue
  integer            ,intent(in) ,optional :: ix(:)
  character(len=*)   ,intent(in) ,optional :: model
  integer            ,intent(in) ,optional :: run_type
  integer            ,intent(in) ,optional :: run_class
  character(len=*)   ,intent(in) ,optional :: initial_date
  integer            ,intent(in) ,optional :: forecast_time
  integer            ,intent(in) ,optional :: ens_member
  integer            ,intent(in) ,optional :: exp_id
```

write_fdbk_file

Write the content of the derived type variable `fb` to the file.

```
type(t_fdbk_data) ,intent(inout)      :: fb      ! feedback file data type
character(len=*)   ,intent(in) ,optional :: file   ! file name
character(len=*)   ,intent(in) ,optional :: comment ! history extension
character(len=*)   ,intent(in) ,optional :: opt     ! options: model, obstypes
logical            ,intent(in) ,optional :: close  ! if .false. dont close
```

destruct

deallocate components of derived type `t_fdbk_data`.

```
subroutine destruct_fdbk_data (data)
  type (t_fdbk_data) ,intent(inout) :: data
```

5.2.4. Feedback file output for radiances (mo_fdbk_rad)

Simple interface to write radiance feedback files based on functionality in module `mo_fdbk_io` (section 5.2.3). An example for its usage is given by program `test_fdbk_rad` (5.3.3).

Typical usage:

- set up derived type meta data
 1. call `setup_fdbk` ! from module `mo_fdbk`
 call `create_fdbk` ! from module `mo_fdbk`
 call **`setup_fdbk_rad`** ! from this module
- fill in remaining header and body content
 2. `fdbk data% h% .. =`
`fdbk data% b% .. =`
- write header and body content
 3. call **`write_fdbk_rad`** ! from this module
- optionally write verification data
 4. call `add_verification` ! from module `mo_fdbk`
 call `write_veri` ! from module `mo_fdbk`
- close the file
 5. call `close_fdbk` ! from module `mo_fdbk`
 call `cleanup_fdbk` ! from module `mo_fdbk`

setup_fdbk_rad

Allocates the derived type components of `d` for header and body and pre-sets some data from the subroutine parameters.

```

subroutine setup_fdbk_rad (d, n_fov, n_chan, refdate, reftime, &
                        ident, varno, chan, instr, e_o      )
type(t_fdbk_data) ,intent(inout) :: d
integer                :: n_fov          ! number of FOV
integer                :: n_chan         ! number of channels / fov
integer                :: refdate        ! yyyymmdd
integer                :: reftime        ! hhmm
integer                :: ident          ! WMO satellite id
integer                :: varno (n_chan) ! variable number
integer                :: chan  (n_chan) ! channel number
integer                :: instr (n_chan) ! WMO instrument number
real                  :: e_o  (n_chan) ! observation error

```

write_fdbk_rad

Writes the content of the derived type variable `d` to a file.

```

subroutine write_fdbk_rad (d)
type(t_fdbk_data) ,intent(inout) :: d

```

5.3. Test facilities and 3D-Var interface

5.3.1. Test program (test_feedback)

test_feedback

This program calls the subroutines of module **`mo_fdbk`** to create an (currently empty) feedback file and to write the LaTeX tables used in this document. It may serve as an example program how to write feedback files.

5.3.2. Test program (test_feedback_read)

test_feedback_read

This program reads a feedback file and does some calculation and printout. It may serve as an example program how to read feedback files.

Two different interfaces are used, either the high level interface provided by module **mo_fdbk_io**, or the low level interface from module **mo_fdbk**. The name of the file to read and the interface to be used must be passed by stdin.

5.3.3. Test program (test_fdbk_rad)

test_fdbk_rad

Example program to write a feedback file for radiances, based on modules **mo_fdbk_io** (5.2.3) and **mo_fdbk_rad** (5.2.4).

5.3.4. 3d-Var interface (mo_fdbk_3dvar)

This module holds the feedback file interface used in the 3dvar/LETKF. This module is very 3dvar specific and only included as a reference.

6. Feedback File Example (ncdump -h)

File created by test_feedback :

```
netcdf test_feedback {
dimensions:
    d_hdr = 10 ;
    d_body = 50 ;
    d_radar = 1 ;
    d_veri = UNLIMITED ; // (3 currently)
    d_2 = 2 ;
    d_3 = 3 ;
    char10 = 10 ;
    char12 = 12 ;
    char32 = 32 ;
    char64 = 64 ;
variables:
    int i_body(d_hdr) ;
        i_body:longname = "index of 1st entry in report body" ;
        i_body:_FillValue = -2147483647 ;
    int l_body(d_hdr) ;
        l_body:longname = "number of entries in report body" ;
        l_body:_FillValue = -2147483647 ;
    short n_level(d_hdr) ;
        n_level:longname = "number of levels in report" ;
        n_level:_FillValue = -32767s ;
    short data_category(d_hdr) ;
        data_category:longname = "BUFR4 data category" ;
        data_category:_FillValue = -32767s ;
    short sub_category(d_hdr) ;
```

```

    sub_category:longname = "BUFR4 data sub-category" ;
    sub_category:_FillValue = -32767s ;
short center(d_hdr) ;
    center:longname = "station processing center" ;
    center:_FillValue = -32767s ;
short sub_center(d_hdr) ;
    sub_center:longname = "station processing sub-center" ;
    sub_center:_FillValue = -32767s ;
byte obstype(d_hdr) ;
    obstype:longname = "observation type" ;
    obstype:_FillValue = -127b ;
short codetype(d_hdr) ;
    codetype:longname = "code type" ;
    codetype:_FillValue = -32767s ;
int ident(d_hdr) ;
    ident:longname = "station or satellite id as integer" ;
    ident:_FillValue = -2147483647 ;
char statid(d_hdr, char10) ;
    statid:longname = "station id as character string" ;
float lat(d_hdr) ;
    lat:units = "degree" ;
    lat:longname = "latitude" ;
    lat:_FillValue = 9.96921e+36f ;
float lon(d_hdr) ;
    lon:units = "degree" ;
    lon:longname = "longitude" ;
    lon:_FillValue = 9.96921e+36f ;
short time(d_hdr) ;
    time:units = "min" ;
    time:longname = "observation minus reference time" ;
    time:_FillValue = -32767s ;
short time_nomi(d_hdr) ;
    time_nomi:units = "min" ;
    time_nomi:longname = "nominal (synoptic) minus reference time" ;
    time_nomi:_FillValue = -32767s ;
short time_dbase(d_hdr) ;
    time_dbase:units = "min" ;
    time_dbase:longname = "data base minus reference time" ;
    time_dbase:_FillValue = -32767s ;
int z_station(d_hdr) ;
    z_station:units = "m" ;
    z_station:longname = "station height" ;
    z_station:_FillValue = -32767 ;
short z_modsurf(d_hdr) ;
    z_modsurf:units = "m" ;
    z_modsurf:longname = "model surface height" ;
    z_modsurf:_FillValue = -32767s ;
byte r_state(d_hdr) ;
    r_state:longname = "status of the report" ;
    r_state:_FillValue = -127b ;
int r_flags(d_hdr) ;
    r_flags:longname = "report quality check flags" ;
    r_flags:_FillValue = -2147483647 ;
byte r_check(d_hdr) ;
    r_check:longname = "check which raised the report status flag value" ;
    r_check:_FillValue = -127b ;
byte sta_corr(d_hdr) ;
    sta_corr:longname = "station correction indicator" ;
    sta_corr:_FillValue = -127b ;

```

```

int index_x(d_hdr) ;
    index_x:longname = "index x of model grid point assigned to report" ;
    index_x:_FillValue = -2147483647 ;
short index_y(d_hdr) ;
    index_y:longname = "index y of model grid point assigned to report" ;
    index_y:_FillValue = -32767s ;
byte mdlsfc(d_hdr) ;
    mdlsfc:longname = "model surface characteristics" ;
    mdlsfc:_FillValue = -127b ;
short instype(d_hdr) ;
    instype:longname = "station type or satellite instrument type" ;
    instype:_FillValue = -32767s ;
float sun_zenit(d_hdr) ;
    sun_zenit:longname = "sun zenith angle" ;
    sun_zenit:_FillValue = 9.96921e+36f ;
int obs_id(d_hdr) ;
    obs_id:longname = "unique observation id" ;
    obs_id:_FillValue = -2147483647 ;
byte source(d_hdr) ;
    source:longname = "input file number" ;
    source:_FillValue = -127b ;
int record(d_hdr) ;
    record:longname = "record number in the input file" ;
    record:_FillValue = -2147483647 ;
short subset(d_hdr) ;
    subset:longname = "subset number in the record" ;
    subset:_FillValue = -32767s ;
int dbkz(d_hdr) ;
    dbkz:longname = "DWD data base id" ;
    dbkz:_FillValue = -2147483647 ;
byte index_d(d_hdr) ;
    index_d:longname = "model grid diamond index assigned to report" ;
    index_d:_FillValue = -127b ;
short varno(d_body) ;
    varno:longname = "type of the observed quantity" ;
    varno:_FillValue = -32767s ;
float obs(d_body) ;
    obs:longname = "bias corrected observation" ;
    obs:_FillValue = 9.96921e+36f ;
float bcor(d_body) ;
    bcor:longname = "bias correction, corrected minus observed" ;
    bcor:_FillValue = 9.96921e+36f ;
float level(d_body) ;
    level:longname = "level of observation" ;
    level:_FillValue = 9.96921e+36f ;
short level_typ(d_body) ;
    level_typ:longname = "type of level information" ;
    level_typ:_FillValue = -32767s ;
short level_sig(d_body) ;
    level_sig:longname = "level significance; airep rollangle quality" ;
    level_sig:_FillValue = -32767s ;
byte state(d_body) ;
    state:longname = "status of the observation" ;
    state:_FillValue = -127b ;
int flags(d_body) ;
    flags:longname = "observation quality check flags" ;
    flags:_FillValue = -2147483647 ;
byte check(d_body) ;
    check:longname = "check which raised the observation status flag value" ;

```

```

        check:_FillValue = -127b ;
float e_o(d_body) ;
    e_o:longname = "observational error" ;
    e_o:_FillValue = 9.96921e+36f ;
short qual(d_body) ;
    qual:longname = "observation confidence from data provider" ;
    qual:_FillValue = -32767s ;
float plevel(d_body) ;
    plevel:units = "Pa" ;
    plevel:longname = "nominal pressure level" ;
    plevel:_FillValue = 9.96921e+36f ;
float veri_data(d_veri, d_body) ;
    veri_data:longname = "modelled quantity (as indicated by veri_ens_member)" ;
    veri_data:_FillValue = 9.96921e+36f ;
char veri_model(d_veri, char10) ;
    veri_model:longname = "model used for verification, e.g. COSMO, GME ..." ;
byte veri_run_type(d_veri) ;
    veri_run_type:longname = "type of model run" ;
    veri_run_type:_FillValue = -127b ;
byte veri_run_class(d_veri) ;
    veri_run_class:longname = "class of model run" ;
    veri_run_class:_FillValue = -127b ;
char veri_initial_date(d_veri, char12) ;
    veri_initial_date:units = "yyyymmddhhmm" ;
    veri_initial_date:longname = "start of verification period" ;
int veri_forecast_time(d_veri) ;
    veri_forecast_time:units = "hhmm" ;
    veri_forecast_time:longname = "forecast time at verification_ref_time" ;
    veri_forecast_time:_FillValue = -2147483647 ;
float veri_resolution(d_veri, d_2) ;
    veri_resolution:units = "degree" ;
    veri_resolution:longname = "model resolution, seperately for the x- and y-dimension" ;
    veri_resolution:_FillValue = 9.96921e+36f ;
int veri_domain_size(d_veri, d_3) ;
    veri_domain_size:longname = "domain size: nx,ny,nz for COSMO; ni,ni,nz for GME/ICON" ;
    veri_domain_size:_FillValue = -2147483647 ;
char veri_description(d_veri, char64) ;
    veri_description:longname = "more detailed description than veri_run_type" ;
int veri_ens_member(d_veri) ;
    veri_ens_member:longname = "ensemble member number, special meaning for non-positive values" ;
    veri_ens_member:_FillValue = -2147483647 ;
int veri_exp_id(d_veri) ;
    veri_exp_id:longname = "experiment Id of the verification run" ;
    veri_exp_id:_FillValue = -2147483647 ;
int veri_operator_flag(d_veri) ;
    veri_operator_flag:longname = "observation operator flags" ;
    veri_operator_flag:_FillValue = -2147483647 ;

// global attributes:
:history = "202604011520 3DVAR 200701010000 analysis
           "202604011520 3DVAR 200612310000 24 h forecast
           " ;
:title = "3DVAR      Verification Data" ;
:institution = "German Weather Service" ;
:source = "3DVAR      01.00" ;
:file_version_number = " 1.02" ;
:n_hdr = 0 ;
:n_body = 0 ;
:n_radar = 0 ;

```



```
    :verification_ref_date = 20070101 ;  
    :verification_ref_time = 0 ;  
    :verification_start = 0 ;  
    :verification_end = 0 ;  
    :resolution = 1.f, 1.f ;  
    :domain_size = 192, 192, 40 ;  
}
```

A. Filename conventions for feedback files

The technical specification of the feedback file format is used for the following types of files used at DWD, distinguishable by prefix (and by convention consisting of 3 letters):

- cof** Feedback file from deterministic variational data assimilation containing only accepted and active variables.
- mon** Like **cof**, but in addition containing also variables with lower status which can be used for monitoring and verification purposes.
- ekf** Feedback file from LETKF. May contain active variables and other with lower status.
- fof** ("feedback observation file"): Feedback file from online observation operators during model run (e.g., within COSMO or ICON-LAM). Contains model-equivalents from deterministic model run or single ensemble member.
- mof** ("model-equivalent observation file"): Similar to **fof**, but may hold model-equivalents of deterministic and all ensemble members in a single file.
- ver** ("verification observation file"): Generated by MEC using **mon** or **cof** as input. Contains verification data from multiple deterministic model forecasts (different lead times, different models, different experiments).
- vrn** Like **ver**, but for nest domains of ICON.
- vep** Like **ver**, but for ensemble forecasts (possibly including deterministic control run). Input data may be taken from either **mon**, **cof**, **ekf** or from observational data processed by MEC.
- vpn** Like **vep**, but for nest domains of ICON.
- pof** ("preprocessed observation file"): This format is reserved for intermediate observational data after a preprocessing step, but need not contain any model-related information. The preprocessing shall guarantee a data structure that is consistent with expectations by a subsequent use of the data assimilation code.
- rof** ("raw observation file"): Similar to **pof**, this format is intended for observational data only. No model information is expected. Files using this format may have been generated by tools that have not been fully validated and will be treated like unprocessed observational data that need full checking.

The **rof** format may be useful for data producers not having the capability to generate standard WMO formats (e.g. BUFR), for testing data from new instruments where no exchange format has been standardized, and for experimental purposes. The successful use of **rof** may require introduction of new variables not defined in this document and adaptation of the feedback reading code.